

File Input and Output in C/C++

Goal: Work with students to process file input/output in C/C++, working with structures and pointers.

Refreshener:

- We can open a file with `fopen` function.
Syntax: `FILE *fp = fopen("{filepath}", "m");`
`m` is a file mode and can be:
 - "a": for appending to a file
 - "w": for writing to a file
 - "r": for reading from a file
- **Always check if `fopen` did not return NULL.**
- You can read from a file using `fscanf` or `fgets`
 - `int fscanf(FILE *fp, const char *format, ...)` - used to read structured input files with a specified format.
 - `char* fgets(const char* str, unsigned int n, FILE *fp)` - Used to read a file line by line.
 - You can search online for signatures of these functions if you don't remember.
- You can write to a file using the `fprintf` function
`int fprintf(FILE *fp, const char *format, ...)`
- **Always close the files you opened**
`fclose(FILE *fp)`

Working with Structures

What are Structures?

These are custom data types you can declare in both C and C++. You can use them to represent complex ideas.

```
struct Student {  
    int age;  
    char *name;  
    float height;  
};
```

Structs are defined in the interface file.

You can then declare variables and give it the type of your defined structure:

```
struct Student dsaStudent;
```

You can also initialize it right away

```
struct Student dsaStudent = {24, "Cool Kid", 1.45};
```

----- **Practice: getting started with I/O to files and basic sorting algorithms**

Given a file containing student records with the following format:

The first line contains the number of students, followed by one student per line in the format (id : number, name: char[] mark : float)

A student can be defined as

```
struct Student {  
    int id;  
    char name[100];  
    float mark;  
};
```

Example :

```
3  
1 Eric 16.7  
2 Sadio 10.5  
3 Mane 15
```

Write a function that takes in parameters the input file path (input.txt), allocates enough space for an array to store all the students. Students should be sorted in ascending order by mark.

Output the sorted list to a file (output.txt).

You will need to write these functions

Read number of students

```
int read_student_count(FILE *fp);
```

Read all student records

The number of students will not exceed 1,000

```
void read_students(FILE *fp, Student* students, int count);
```

Sorting algorithm

- Implements the sorting algorithm
- Calls compare_students
- Can be swapped later to make the algorithm stable

```
void sort_students(Student *students, int count);
```

Output / Writing functions

- Write Andrew ID on first line
- Write each student on a new line

- Format with spaces

```
void write_students(
    const char* outputPath,
    const Student *students,
    int count
);
```

Process the input file using these helper functions, The input file should be opened in process_input and the FILE* passed to the reading functions.

```
void process_input(const char inputPath[]);
```

The output for this example should be:

```
Andrewid
2 Sadio 10.5
3 Mane 15
1 Eric 16.7
```

Next Steps for the assignment :

- Make your algorithm stable
- Adapt the Student Data structure to what you need